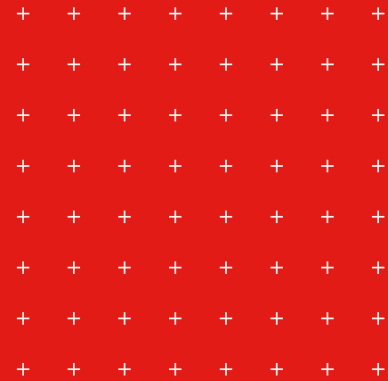
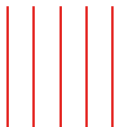




Visite de mi-stage

Maintien en condition de sécurité de
LF Energy SEAPATH





SEAPATH

Présentation

SEAPATH [1] :

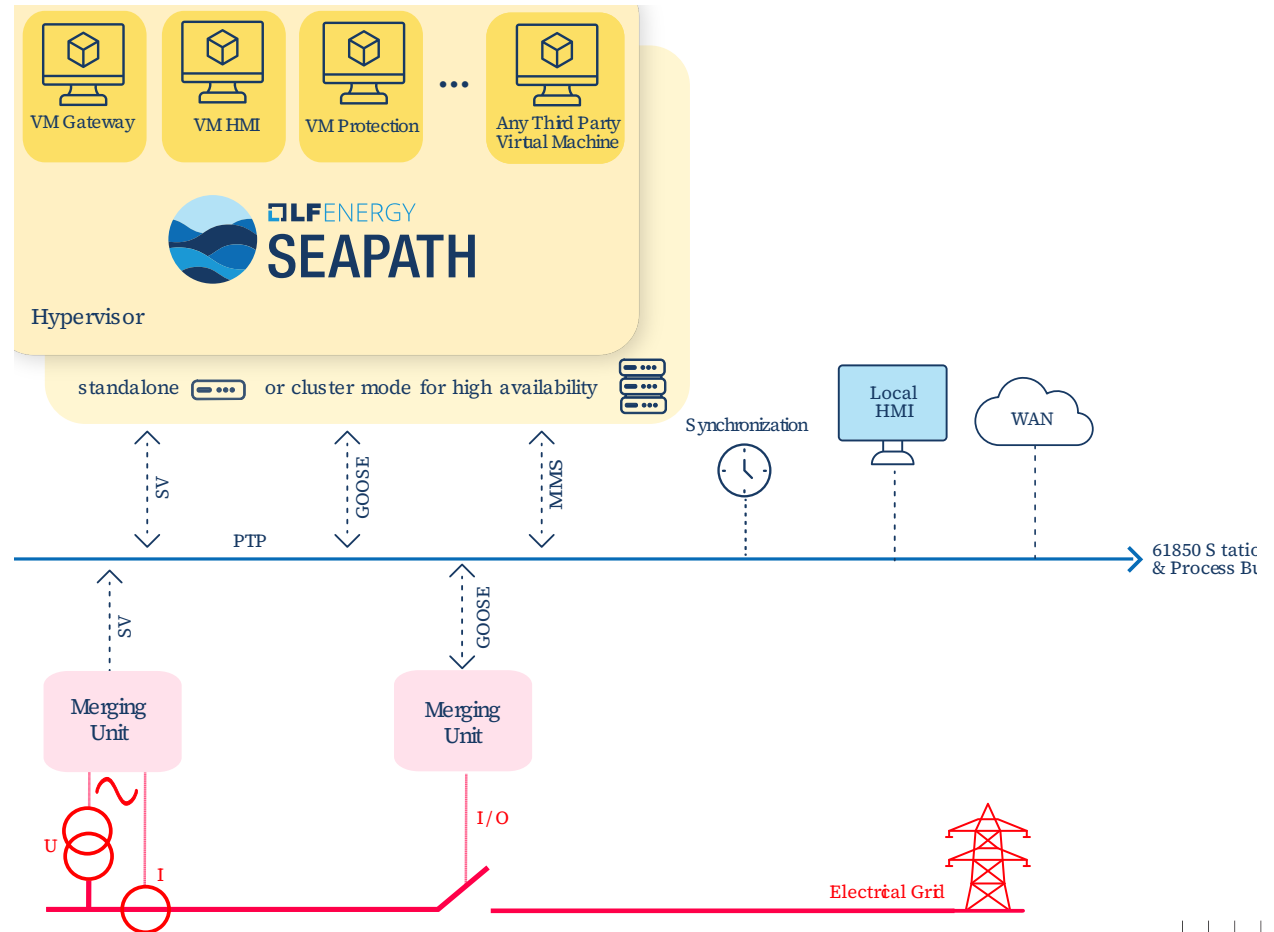
- hyperviseur (VMs)
- temps réel
- haute disponibilité
- basé sur le noyau Linux

Usage :

- postes électriques

2 versions :

- Debian [2]
- Yocto [3]



Problématique du stage

Pas de visibilité sur les CVEs (vulnérabilités) présentes dans SEAPATH.

- mise en place d'outils pour **détecter les vulnérabilités**
 - + utilisation de **SBOMs** (Software Bill Of Materials)
 - + à faire dans les deux versions de SEAPATH
- **analyse des CVEs remontées**
 - + vérifier si applicables
 - + corriger si possible
- mettre en place du **monitoring** pour que cela soit fait en continu
 - + via les CI/CD existantes
- utiliser l'outil **VulnScout** [4] développé par Savoir-faire Linux

Table des matières

- SEAPATH _____ 2
- CVEs sur SEAPATH Yocto _____ 6
- Travail sur VulnScout _____ 14
- CVEs sur SEAPATH Debian _____ 18



CVEs sur SEAPATH Yocto

Détection des CVEs : possibilités

- SEAPATH Yocto = distribution **déclarée en code** (Bitbake)
→ facile de générer un SBOM
- différentes techniques pour détecter les CVEs :
 - + **générer un SBOM** puis utiliser un **outil de matching** sur le SBOM
 - + utiliser l'**outil « cve-check » intégré** à Yocto pour détecter les CVEs
 - ◆ **possiblement avec des scripts de filtrage** supplémentaires
(filtre les CVEs non compilées du noyaux Linux)
- différents technologies :
 - + formats de SBOM : SPDX2, SPDX3, CycloneDX, OpenVEX [5]
 - + outils de matching : Grype [6], sbom-cve-check [7]

Détection des CVEs : travail préliminaire

- Mise en place des outils pour prise en main.
- Pour établir une comparaison, sélection de plusieurs scénarios :
 - + «cve-check» de Yocto avec plus ou moins de scripts de filtrage
 - + Génération de SBOMs (SPDX2 et OpenVEX) puis ingestion dans «sbom-cve-check» pour détection des CVEs
- **Évaluation quantitative** des résultats de ces 3 scénarios :
 - + comparaison du **nombre total de CVEs** remontées
 - + évaluation du **nombre de faux positifs et faux négatifs**
- Sélection de la meilleure solution pour notre cas d'usage
- Écriture d'un rapport à publier ensuite en billet de blog

Détection des CVEs : rapport sur la détection dans Yocto

Savoir-faire Linux

Youenn Le Jeune

CVE detection for Yocto

Vulnerabilities assessment is a crucial aspect of developing and maintaining a software project. If you are using Yocto, there are multiple possible approaches to detect and filter CVEs in a Yocto-built image. This report will compare some of them and present data, analysis and recommendations.

1. Analysis details

- All of the detection runs have been made on the same version of SEAPATH [1] on the 2026-03-10.
- SEAPATH is not based on a linux-yocto kernel but on a custom recipe, using the mainline-linux-rt kernel. Some details are provided in Section 3.
- We are using Yocto Scarthgap, the current LTS version at the time of writing.
- The kernel version is 6.12.66, a very recent revision at the moment the analysis was done.
- Runs have been made with only a few minutes intervals between them, meaning the probability of new CVEs being discovered in-between is slim.

2. Tested tools and scenarios

2.1. "Vanilla" Yocto cve-check

This is the most basic scenario: we inherit the cve-check class from the Poky layer [2], and we include the cve-extra-exclusions.inc file to filter some non-applicable CVEs.

2.2. "Light" meta-vulnscout

Savoir-faire Linux's [3] VulnScout [4] team has developed a Yocto layer that contains a lot of useful classes to improve the vanilla cve-check results: meta-vulnscout [5].

In this scenario, we will only use 2 of those classes: kernel_generate_cve_exclusions and improve_kernel_cve_report, which are based on similarly-named scripts in the OpenEmbedded-Core layer (more details on meta-vulnscout GitHub page).

2.3. "Full" meta-vulnscout

This scenario is based on the previous one, with the addition of the kernel_filter_nonbuilt_cves class crafted by the VulnScout team.

2.4. "External" sbom-cve-check

The Bootlin [6] team has created sbom-cve-check [7], an util to detect CVEs from SBOM files. It can work with files generated by a Yocto build, without having to run during the build process. We are using the create-spxd-2.2 and vex classes to generate the input files.

3. Details on the Linux Kernel recipe

The linux-yocto kernel recipe references some extra exclusion files' that filter a lot of kernel CVEs by default. Since SEAPATH is using a custom kernel recipe, we had to integrate those utils in our recipe, otherwise we would have got a lot of false positives.

One of those 2 exclusion files, cve-exclusion_6.18.inc, is generated by a script using the Linux Kernel "vulns" repository [8] as a source. We can ignore it since it's already a data source of our tools, as we are going to see later. The problem lies in the other file, cve-exclusion.inc: it contains CVE statuses for CVEs that are *not* completely documented in the vulns repository. Therefore, we

¹The cve-exclusion files from <https://git.openembedded.org/openembedded-core/tree/meta/recipes-kernel/linux?id=6798bd73ab2693bae6a7e0425e2519250fdcc8&h=scarthgap>

Savoir-faire Linux

Youenn Le Jeune

integrated this exclusion file in SEAPATH, after splitting it to suit our kernel version needs. The commit for this work is available on GitHub: [SEAPATH/meta-scapath@b9c1fc1](https://github.com/SEAPATH/meta-scapath@b9c1fc1).

One last thing we had to do is to make the built kernel sources included in the generated SPDX file, so sbom-cve-check can filter non-built kernel CVEs based on that. In linux-yocto it's simply a matter of enabling the SPDX_INCLUDE_COMPILED_SOURCES variable, but it relies on some custom kernel configuration that is present by default in linux-yocto and that we had to include in our recipe. The commit that enabled this is also available on GitHub: [SEAPATH/meta-scapath@abd2818](https://github.com/SEAPATH/meta-scapath@abd2818).

4. Results

4.1. Total amounts

The first interesting thing to notice in Figure 1 and Table 1 is the total volume of CVEs detected by the tools: in comparison with the vanilla cve-check tool, both the meta-vulnscout and the sbom-cve-check utils detected roughly 2000 more CVEs. This is easily explainable since cve-check's only database is the NVD [2] which is not complete, especially concerning the Linux kernel-related CVEs. meta-vulnscout classes uses the same data as cve-check *plus* the cvelistV5 database [10] and the official kernel "vulns" repository [8]. On the other hand, sbom-cve-check only uses cvelistV5 and the NVD database (but cvelistV5 should include all CVEs from the kernel vulns*). A summary is available in Table 2.

	fixed	not_affected	under_investigation	total
vanilla	14957	95	131	15 183
light	16 534	378	363	17 275
full	16 534	453	287	17 274
external	16 520	403	120	17 043

Table 1: Raw CVE statuses data

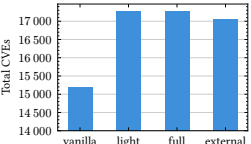


Figure 1: Total amount of detected CVEs, per detection scenario.

Beware, the y axis does not start at 0.

	NVD	cvelistv5	vulns	
cve-check	✓	✗	✗	Fails to link some CVEs with recipes
meta-vulnscout	✓	✓	✓	Exports everything
sbom-cve-check	✓	✓	✗	Does not export rejected CVEs

Table 2: Summary of CVE origin databases

²<https://www.cve.org/PartnerInformation/ListoPartners/partner/Linux>

Savoir-faire Linux

Youenn Le Jeune

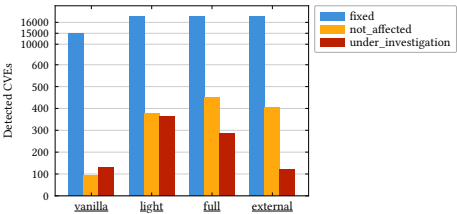


Figure 2: Statues of the detected CVEs, per detection scenario.

Mind the y axis for the large "fixed" amounts.

Another thing to notice in the total amount of detected CVEs is that sbom-cve-check has detected ~230 less CVEs than meta-vulnscout. It is actually because the former automatically discards CVEs that have been rejected by CNAs (such as CVE-2021-47084) whereas the latter keeps them as "not affected", thus its total amount is bigger.

4.2. Assessment size

The most interesting data to us is the amount of CVEs that are "under investigation". It tells us how much time we should spend on our assessment. The smaller the better, as long as it does not have false negatives.

We can already say that the vanilla cve-check contains a lot of false negatives since it's based on only 1 incomplete database, as we have seen in the previous section.

Concerning the meta-vulnscout scenarios, it is pretty clear that adding the kernel_filter_nonbuilt_cves class decreases a lot the amount of CVEs to assess: as shown in Table 1, the full scenario has ~20% less CVEs in the "under investigation" status than the light one. And this is for a recent kernel version, for ancient versions it can be way higher: we made the same comparison for a version of SEAPATH that is more than 1 year old (SEAPATH 1.0.0 from January 22th, 2025) and we get 70% less CVEs to investigate! See the raw data in Table 3.

	fixed	not_affected	under_investigation	total
vanilla	12 524	81	2552	15 157
full	13 925	1670	765	16 360

Table 3: Raw CVE statuses data for SEAPATH 1.0.0

sbom-cve-check produces even less CVEs to assess. From our analysis, it seems to be thanks to its better filtering on non-built kernel CVEs (e.g. CVE-2025-40338 which sbom-cve-check correctly flags as "compiled code not present" whereas meta-vulnscout does not). In terms of CVEs reported, no matter their statuses, it looks like sbom-cve-check includes *all* the CVEs from meta-vulnscout with some more. You can see the differences in assessment between the full and external scenario in details in Section 6.2.

Unfortunately we could not try sbom-cve-check on the old SEAPATH 1.0.0 as we did with meta-vulnscout since it requires the vex class, not present in the Yocto Scarthgap revision in use in 1.0.0.

Analyse des CVEs

- Quantité de CVEs potentiellement applicable assez importante (~ 130)
- **Choix de la portée** de l'analyse faite dans SEAPATH :
 - + CVEs critiques ($CVSS \geq 9.0$)
 - + CVEs importantes et facilement exploitables ($CVSS \geq 9.0 \wedge EPSS \geq 50\%$)
- Avec ces critères, reste ~ 15 CVEs à analyser
- La plupart sont des faux positifs :
 - + Dépendent de **configuration spécifiques non présentes** dans SEAPATH
 - ♦ Ajout d'exceptions dans le code de SEAPATH
 - + **Pas de version cible** dans les bases de données publiques de CVEs
 - ♦ Correction et contribution au Yocto Project via les *mailing lists*

Intégration en CI

Intérêts :

- rendre publique la liste de CVEs détectées
- faire échouer les PR (Pull Requests) lorsqu'elles rajoutent des vulnérabilités

Travail effectué :

- réécriture de la CI (Jenkins [8] → GitHub Actions [9])
 - + gros travail de synchronisation entre les différents dépôts
- détection des CVEs dans la CI avec la méthode sélectionnée précédemment
- mise en place de lancements quotidiens pour chercher nouvelles CVEs
- feedback sur les PR en fonction des résultats de la CI

Intégration en CI

Actions

All workflows

- Build (internal)
- Build on push
- CVE check (internal)
- Periodic CVE check
- Pull Requests
- Pull Requests Summary

Management

- Caches
- Attestations
- Usage metrics
- Performance metrics

All workflows

Showing runs from all workflows

Filter workflow runs

40 workflow runs			Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✓	Pull Requests Summary					Apr 2, 9:49 AM GMT+2	...
	Pull Requests Summary #3: completed by eroussy					35s	
✗	PR #77 at seapath/yocto-bsp	dupremathieu:cqfd-docker-...				Apr 2, 9:10 AM GMT+2	...
	Pull Requests #5: Pull request #77 synchronize by eroussy					39m 26s	
✓	Pull Requests Summary					Apr 1, 3:04 PM GMT+2	...
	Pull Requests Summary #2: completed by eroussy					32s	
✓	Pull Requests Summary					Apr 1, 2:59 PM GMT+2	...
	Pull Requests Summary #1: completed by eroussy					28s	
✗	PR #77 at seapath/yocto-bsp	dupremathieu:cqfd-docker-...				Apr 1, 2:59 PM GMT+2	...
	Pull Requests #4: Pull request #77 synchronize by eroussy					33s	
✗	Periodic CVE check	scarthgap				Apr 1, 10:41 AM GMT+2	...
	Periodic CVE check #6: Scheduled					6m 53s	

Intégration en CI



seapath-cicd (Bot) commented 3 weeks ago

Build

SUCCESS: all flavors compiled correctly.

CVE Check

CVE DETECTED: some vulnerabilities triggered the fail condition.

See details in the [workflow run logs](#).



Note

This is an automated CI report. See the [associated workflow run](#)



Annotations

1 error and 4 warnings



Process results

Vulnerabilities triggered the fail condition for your build.



VulnScout host_standalone_efi

Some vulnerabilities triggered fail condition:

Total failed vulnerabilities: 2

= CVE-2025-69720

Severity: CRITICAL

CVSS Score: 9.8

EPSS Score: 0.02%

Affected Packages:

- ncurses (6.4)

= CVE-2026-4176

Severity: CRITICAL

CVSS Score: 9.8

EPSS Score: 0.02%

Affected Packages:

- perl (5.38.4)

[Show less](#)



Travail sur VulnScout

Présentation

- Outil d'**évaluation de vulnérabilités**
- Développé par Savoir-faire Linux (équipe à Montréal), open-source
- Récent : annoncé en mars 2025, toujours en version bêta
- Versatile : supporte de nombreux formats de SBOMs
- Très intégré dans Yocto

Relation avec SEAPATH :

- Utilisé pour évaluer les CVEs
- Utilisé dans la CI:
 - + **génération de rapports** sur les CVEs
 - + vérification des **CVEs critiques** (condition établie précédemment)



VulnScout.io

Contributions

- Bloquants au fil de l'utilisation pour SEAPATH :
 - + ajout de **fonctionnalités manquantes**
 - + correction de **bugs**
- Amélioration de la base de code
 - + typage
 - + linting
- **Revue de code** pour les autres personnes de l'équipe
- Participation aux **réunions hebdomadaires** et aux planifications des cycles de développement

Comparaison avec Dependency-Track

- Autre logiciel open-source de vérifications de vulnérabilités
- Existe depuis plus de 10 ans
- Qu'est-ce que VulnScout peut apporter ?
 - + Déploiement d'une instance
 - + Expérimentations dessus
 - + Comparaison avec VulnScout
 - + Écriture d'un rapport



CVEs sur SEAPATH Debian

Détection des CVEs : possibilités

- Distribution construite sur base Debian
 - + liste de paquets à installer
 - + scripts et fichiers de configuration
 - + images de conteneurs à intégrer
 - + $\Rightarrow$ impossible de générer un SBOM à partir des sources comme avec Yocto
- Autre approche : **générer SBOM à partir du système de fichiers**
 - + utilisation de Syft [10]
- Deux possibilités :
 1. construire image $\rightarrow$ déployer sur une machine $\rightarrow$ lancer Syft
 2. intégrer Syft **dans la construction de l'image**
 - + plus complexe mais plus pratique
 - + solution retenue

Détection des CVEs : travail

- Création d'un *hook* pour faire le scan des fichiers durant la construction
- Modifications dans le script de construction de l'image
 - + Refactoring
 - + Intégration des images conteneurs
 - + Récupération des fichiers de scans créés

Reste à faire :

- Corriger les anciens scripts pour qu'ils restent fonctionnels
- Analyser le SBOM et regarder les CVEs détectées
- Intégration dans la CI

Bibliographie

- [1] LF Energy, « SEAPATH ». Consulté le: 21 avril 2026. [En ligne]. Disponible sur: <https://lfenergy.org/projects/seapath/>
- [2] The Debian Project, « Debian Reference ». Consulté le: 21 avril 2026. [En ligne]. Disponible sur: <https://www.debian.org/doc/manuals/debian-reference/>
- [3] The Yocto Project®, « Yocto Project Documentation ». Consulté le: 21 avril 2026. [En ligne]. Disponible sur: <https://docs.yoctoproject.org/>
- [4] « VulnScout: The Easy-to-Use Tool for Securing Embedded Systems and Staying Compliant », Savoir-faire Linux. Consulté le: 23 avril 2026. [En ligne]. Disponible sur: <https://blog.savoirfairelinux.com/en-ca/2025/vulnscout-the-easy-to-use-tool-for-securing-embedded-systems-and-staying-compliant/>
- [5] M. Mirakhorli *et al.*, « A Landscape Study of Open Source and Proprietary Tools for Software Bill of Materials (SBOM) ». [En ligne]. Disponible sur: <https://arxiv.org/abs/2402.11151>
- [6] Trex Team, *Grype in Production*. NobleTrex Press, 2026. Consulté le: 21 avril 2026. [En ligne]. Disponible sur: https://books.google.com/books/about/Grype_in_Production.html?hl=fr&id=LJ_KEQAAQBAJ

- [7] bootlin, *sbom-cve-check*. Consulté le: 21 avril 2026. [En ligne]. Disponible sur: <https://github.com/bootlin/sbom-cve-check/>
- [8] V. Armenise, « Continuous Delivery with Jenkins: Jenkins Solutions to Implement Continuous Delivery », in *2015 IEEE/ACM 3rd International Workshop on Release Engineering*, 2015, p. 24-27. doi: 10.1109/RELENG.2015.19.
- [9] C. Chandrasekara et P. Herath, *Hands-on GitHub Actions: Implement CI/CD with GitHub Action Workflows for Your Applications*. Berkeley, CA: Apress, 2021. doi: 10.1007/978-1-4842-6464-5.
- [10] *Syft*. Consulté le: 23 avril 2026. [En ligne]. Disponible sur: <https://github.com/anchore/syft>